

Arrays, objetos y modelado de datos en JavaScript

Enfoque de la sesión

En esta sesión vas a trabajar uno de los núcleos más importantes de JavaScript moderno: **representar información de forma estructurada**. El objetivo no es solo aprender qué es un array o qué es un objeto, sino comprender cómo se organizan los datos en un programa y cómo prepararlos para recorrerlos, transformarlos, validarlos y, más adelante, renderizarlos en interfaz.

1. Introducción

Hasta ahora has trabajado con variables, operadores, condicionales, bucles y funciones. Esa base te permite escribir lógica, pero todavía falta una pieza esencial: **cómo representar conjuntos de información de manera útil**.

En una aplicación real rara vez trabajas con un único valor aislado. Lo habitual es manejar:

- una lista de productos;
- un conjunto de usuarios;
- varias tareas con distinto estado;
- una colección de películas;
- un pedido con cliente, fecha, total y líneas de detalle.

Para modelar ese tipo de información, JavaScript utiliza sobre todo dos estructuras:

- **arrays**, cuando quieres almacenar una colección ordenada de elementos;
- **objetos**, cuando quieres representar una entidad con propiedades con nombre.

Saber distinguir ambos casos y combinarlos correctamente es una habilidad fundamental. De ello dependen después tareas como:

- recorrer información;
- buscar valores concretos;
- validar datos;
- transformar resultados;
- enviarlos o recibirlos en formato JSON;
- renderizarlos en el DOM;
- utilizarlos en React como estado, props o datos remotos.

En esta sesión vas a trabajar, por tanto, no solo sintaxis, sino también una forma de pensar: **cómo dar forma a la información para que el programa pueda utilizarla con claridad.**

2. Arrays: colecciones ordenadas de elementos

2.1. Qué es un array

Un array es una estructura que permite almacenar varios valores en una única variable, manteniendo un orden.

```
const frutas = ["manzana", "pera", "plátano"];
```

En este caso, `frutas` contiene tres elementos.

2.2. Cuándo usar un array

Conviene usar un array cuando necesitas representar:

- una lista;
- una secuencia;
- una colección de elementos del mismo tipo o con función parecida;
- datos que vas a recorrer, filtrar, mostrar o transformar.

Ejemplos habituales:

- nombres de estudiantes;
- productos de un catálogo;
- identificadores;
- resultados de una búsqueda;
- tareas pendientes.

2.3. Acceso por índice

Cada elemento ocupa una posición numérica, comenzando en `0`.

```
const frutas = ["manzana", "pera", "plátano"];

console.log(frutas[0]); // "manzana"
console.log(frutas[1]); // "pera"
console.log(frutas[2]); // "plátano"
```

Idea clave

Los arrays están **indexados desde 0**.

Este detalle es simple, pero provoca muchos errores al comenzar.

2.4. Longitud de un array

La propiedad `.length` indica cuántos elementos contiene.

```
const frutas = ["manzana", "pera", "plátano"];

console.log(frutas.length); // 3
```

2.5. Modificación de elementos

Puedes cambiar el valor de una posición concreta.

```
const frutas = ["manzana", "pera", "plátano"];
frutas[1] = "uva";

console.log(frutas); // ["manzana", "uva", "plátano"]
```

2.6. Arrays con distintos tipos

JavaScript permite arrays con elementos de tipos distintos.

```
const mezcla = ["Ana", 25, true, null];
```

Aunque esto es posible, en la práctica conviene que los arrays tengan cierta coherencia estructural para facilitar su uso y mantenimiento.

Referencias específicas del subtema

- MDN Web Docs — Indexed collections
- MDN Web Docs — `Array`
- MDN Web Docs — `length`

3. Recorrido básico de arrays

3.1. Recorrer un array con `for`

Puedes recorrer un array utilizando un índice.

```
const frutas = ["manzana", "pera", "plátano"];

for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

3.2. Recorrer un array con `for...of`

Cuando no necesitas el índice y solo quieres acceder al valor, `for...of` suele ser más claro.

```
const frutas = ["manzana", "pera", "plátano"];

for (const fruta of frutas) {
  console.log(fruta);
}
```

3.3. Criterio de uso

- usa `for` si necesitas el índice;
- usa `for...of` si te interesa directamente cada elemento.

Referencias específicas del subtema

- MDN Web Docs — `for`
- MDN Web Docs — `for...of`

4. Objetos: entidades con propiedades

4.1. Qué es un objeto

Un objeto es una estructura que agrupa información relacionada mediante pares **clave-valor**.

```
const usuario = {
  nombre: "Laura",
  edad: 28,
  activo: true
};
```

Aquí el objeto `usuario` representa una entidad con varias propiedades.

4.2. Cuándo usar un objeto

Conviene usar un objeto cuando quieres representar:

- una persona;
- un producto;
- una tarea;
- una reserva;
- una factura;
- cualquier entidad con atributos identificables.

4.3. Acceso a propiedades

Puedes acceder a una propiedad mediante notación de punto.

```
const usuario = {  
  nombre: "Laura",  
  edad: 28,  
  activo: true  
};  
  
console.log(usuario.nombre);  
console.log(usuario.edad);
```

También existe la notación con corchetes.

```
console.log(usuario["activo"]);
```

4.4. Modificación de propiedades

```
const usuario = {  
  nombre: "Laura",  
  edad: 28,  
  activo: true  
};  
  
usuario.edad = 29;  
usuario.activo = false;  
  
console.log(usuario);
```

4.5. Añadir nuevas propiedades

```
const usuario = {  
  nombre: "Laura",  
  edad: 28  
};  
  
usuario.email = "laura@email.com";
```

4.6. Eliminar propiedades

```
const usuario = {  
  nombre: "Laura",  
  edad: 28,  
  activo: true  
};  
  
delete usuario.activo;  
console.log(usuario);
```

Precaución de diseño

Que una operación sea posible no significa que sea la mejor opción en todos los casos. En aplicaciones más estructuradas suele convenir construir nuevos objetos o controlar mejor los cambios, en lugar de modificar datos de forma desordenada.

Referencias específicas del subtema

- MDN Web Docs — Working with objects
- MDN Web Docs — Object basics

5. Diferencia conceptual entre array y objeto

Esta distinción debe quedar muy clara desde el principio.

Estructura	Mejor uso habitual	Ejemplo
Array	colección ordenada de elementos	lista de productos

Estructura	Mejor uso habitual	Ejemplo
Objeto	entidad con propiedades con nombre	un producto concreto

Observa este ejemplo:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 39.99
};

const productos = [
  {
    id: 1,
    nombre: "Teclado",
    precio: 39.99
  },
  {
    id: 2,
    nombre: "Ratón",
    precio: 19.99
  }
];
```

- `producto` es **un objeto**;
- `productos` es **un array de objetos**.

Idea clave

En aplicaciones reales es muy frecuente trabajar con **arrays de objetos**.
Es decir, colecciones de entidades estructuradas.

Referencias específicas del subtema

- MDN Web Docs — Indexed collections
- MDN Web Docs — Working with objects

6. Datos anidados

6.1. Objetos dentro de objetos

Es habitual que una propiedad contenga otro objeto.

```
const usuario = {
  nombre: "Sofía",
  direccion: {
    calle: "Gran Vía",
    ciudad: "Madrid",
    codigoPostal: "28013"
  }
};

console.log(usuario.direccion.ciudad);
```

6.2. Arrays dentro de objetos

```
const usuario = {
  nombre: "Sofía",
  intereses: ["lectura", "cine", "viajes"]
};

console.log(usuario.intereses[0]);
```

6.3. Objetos dentro de arrays

```
const tareas = [
  { titulo: "Estudiar JavaScript", completada: false },
  { titulo: "Practicar condicionales", completada: true }
];

console.log(tareas[1].titulo);
```

6.4. Importancia del dato anidado

Trabajar con datos anidados es imprescindible porque muchas estructuras reales no son planas. Cuanto antes te acostumbres a leerlas con claridad, más fácil te resultará trabajar con APIs, formularios complejos y componentes que reciban información estructurada.

Referencias específicas del subtema

- MDN Web Docs — Working with objects
- MDN Web Docs — Indexed collections

7. Desestructuración de arrays

7.1. Qué es la desestructuración

La desestructuración permite extraer valores de un array u objeto y asignarlos directamente a variables.

7.2. Ejemplo básico con array

```
const colores = ["rojo", "verde", "azul"];

const [primero, segundo, tercero] = colores;

console.log(primero); // "rojo"
console.log(segundo); // "verde"
console.log(tercero); // "azul"
```

7.3. Saltar posiciones

```
const colores = ["rojo", "verde", "azul"];

const [primero, , tercero] = colores;

console.log(primero);
console.log(tercero);
```

7.4. Valores por defecto

```
const colores = ["rojo"];

const [primero, segundo = "sin definir"] = colores;

console.log(primero); // "rojo"
console.log(segundo); // "sin definir"
```

Referencias específicas del subtema

- MDN Web Docs — Destructuring assignment
-

8. Desestructuración de objetos

8.1. Ejemplo básico

```
const usuario = {  
  nombre: "Irene",  
  edad: 31,  
  ciudad: "Valencia"  
};  
  
const { nombre, edad } = usuario;  
  
console.log(nombre);  
console.log(edad);
```

8.2. Renombrar variables

```
const usuario = {  
  nombre: "Irene",  
  edad: 31  
};  
  
const { nombre: nombreUsuario, edad: edadUsuario } = usuario;  
  
console.log(nombreUsuario);  
console.log(edadUsuario);
```

8.3. Valores por defecto

```
const usuario = {  
  nombre: "Irene"  
};  
  
const { nombre, rol = "usuario" } = usuario;  
  
console.log(nombre);  
console.log(rol);
```

8.4. Utilidad práctica

La desestructuración mejora mucho la legibilidad cuando trabajas con objetos y propiedades conocidas, especialmente al procesar datos, recibir argumentos o construir funciones más expresivas.

Referencias específicas del subtema

- MDN Web Docs — Destructuring assignment
-

9. Operadores `spread` y `rest`

9.1. `spread` en arrays

El operador `spread` permite expandir elementos.

```
const base = [1, 2, 3];
const ampliado = [...base, 4, 5];

console.log(ampliado); // [1, 2, 3, 4, 5]
```

También puede usarse para copiar superficialmente un array.

```
const original = ["a", "b", "c"];
const copia = [...original];
```

9.2. `spread` en objetos

```
const usuario = {
  nombre: "Marta",
  edad: 24
};

const usuarioActualizado = {
  ...usuario,
  ciudad: "Sevilla"
};

console.log(usuarioActualizado);
```

9.3. Sobrescribir propiedades

```
const producto = {
  nombre: "Monitor",
  precio: 199
};
```

```
const productoOferta = {
  ...producto,
  precio: 149
};

console.log(productoOferta);
```

9.4. rest en desestructuración

rest permite agrupar los elementos o propiedades restantes.

```
const numeros = [10, 20, 30, 40];
const [primero, ...resto] = numeros;

console.log(primero); // 10
console.log(resto);   // [20, 30, 40]
```

```
const usuario = {
  nombre: "Lucía",
  edad: 27,
  ciudad: "Bilbao"
};

const { nombre, ...restoDatos } = usuario;

console.log(nombre);
console.log(restoDatos);
```

Idea clave

spread suele ayudarte a **copiar o combinar** estructuras.

rest suele ayudarte a **recoger el resto** de elementos o propiedades.

Referencias específicas del subtema

- MDN Web Docs — Spread syntax
- MDN Web Docs — Rest parameters

10. optional chaining y acceso seguro a propiedades

10.1. El problema

Cuando una propiedad intermedia no existe, acceder a una ruta completa puede provocar un error.

```
const usuario = {
  nombre: "Elena"
};

// console.log(usuario.direccion.ciudad); // Error
```

10.2. Solución con `optional chaining`

`optional chaining` permite acceder de forma segura a propiedades que podrían no existir.

```
const usuario = {
  nombre: "Elena"
};

console.log(usuario.direccion?.ciudad); // undefined
```

10.3. Cuándo es útil

Es especialmente útil cuando:

- trabajas con datos incompletos;
- recibes información externa;
- no todas las propiedades son obligatorias;
- la estructura puede variar.

Criterio práctico

`optional chaining` no sustituye a entender la estructura de tus datos.

Te ayuda a acceder con seguridad, pero debes seguir sabiendo qué esperas encontrar.

Referencias específicas del subtema

- MDN Web Docs — Optional chaining (`?.`)

11. `nullish coalescing` y valores por defecto

11.1. Qué resuelve

A veces necesitas establecer un valor por defecto cuando una propiedad es `null` o `undefined`.

```
const usuario = {
  nombre: "Raúl",
  ciudad: null
};

const ciudadMostrada = usuario.ciudad ?? "Ciudad no disponible";

console.log(ciudadMostrada);
```

11.2. Diferencia respecto a `||`

El operador `||` toma como falsos más valores de los que a veces te interesa considerar.

```
const cantidad = 0;
const resultado = cantidad || 10;

console.log(resultado); // 10
```

Si `0` es un valor válido, ese comportamiento puede no ser el deseado.

Con `??`, solo se sustituye cuando el valor es `null` o `undefined`.

```
const cantidad = 0;
const resultado = cantidad ?? 10;

console.log(resultado); // 0
```

11.3. Cuándo conviene usarlo

`??` es muy útil cuando:

- `0`, `false` o `""` pueden ser válidos;
- solo quieres cubrir ausencia real de dato.

Referencias específicas del subtema

- MDN Web Docs — Nullish coalescing operator (`??`)
-

12. JSON: intercambio de información estructurada

12.1. Qué es JSON

JSON (*JavaScript Object Notation*) es un formato textual muy utilizado para intercambiar información estructurada entre sistemas.

Se parece a la sintaxis de los objetos de JavaScript, pero no es exactamente lo mismo.

12.2. Ejemplo de texto JSON

```
{  
  "nombre": "Andrea",  
  "edad": 29,  
  "activo": true  
}
```

12.3. Convertir objeto JavaScript a JSON

```
const usuario = {  
  nombre: "Andrea",  
  edad: 29,  
  activo: true  
};  
  
const usuarioJSON = JSON.stringify(usuario);  
  
console.log(usuarioJSON);
```

12.4. Convertir JSON a objeto JavaScript

```
const usuarioJSON = '{"nombre":"Andrea","edad":29,"activo":true}';  
const usuario = JSON.parse(usuarioJSON);  
  
console.log(usuario.nombre);
```

12.5. Cuándo aparece JSON

JSON es central en tareas como:

- guardar información en `localStorage`;
- trabajar con APIs;
- intercambiar datos entre frontend y backend;

- serializar estructuras.

⚠ Precisión importante

Un objeto de JavaScript y un texto JSON no son exactamente lo mismo. JSON es una representación textual. Para trabajar con él como estructura operable en JavaScript necesitas parsearlo.

Referencias específicas del subtema

- MDN Web Docs — JSON
- MDN Web Docs — `JSON.parse()`
- MDN Web Docs — `JSON.stringify()`

13. Modelado de datos: pensar antes de programar

13.1. Qué significa modelar datos

Modelar datos consiste en decidir cómo representar una realidad dentro del programa.

Por ejemplo, si quieres representar una tarea, debes decidir qué información necesita.

```
const tarea = {  
  id: 1,  
  titulo: "Estudiar arrays y objetos",  
  completada: false,  
  prioridad: "alta"  
};
```

13.2. Ejemplo de colección bien modelada

```
const tareas = [  
  {  
    id: 1,  
    titulo: "Estudiar arrays y objetos",  
    completada: false,  
    prioridad: "alta"  
  },  
  {  
    id: 2,
```

```
    titulo: "Practicar desestructuración",  
    completada: true,  
    prioridad: "media"  
  }  
];
```

13.3. Criterios básicos de buen modelado

Un modelado inicial razonable suele buscar:

- propiedades claras y con nombres coherentes;
- estructuras predecibles;
- evitar mezclar significados distintos en la misma propiedad;
- facilitar el recorrido y la transformación posterior.

13.4. Por qué importa tanto

Si modelas mal los datos:

- las búsquedas son más difíciles;
- las validaciones se complican;
- el renderizado posterior se vuelve confuso;
- la lógica del programa pierde claridad.

Si modelas bien los datos, muchas tareas posteriores se simplifican.

Idea clave

Un buen diseño de datos facilita el resto del programa.

Muy a menudo, cuando algo “se complica demasiado”, el problema no está solo en la lógica, sino en cómo has representado la información.

Referencias específicas del subtema

- MDN Web Docs — Working with objects
- MDN Web Docs — Indexed collections
- MDN Web Docs — JSON

14. Copias, referencias y mutación accidental

14.1. El problema de las referencias

Cuando trabajas con arrays y objetos, muchas veces no estás creando una copia nueva, sino otra referencia al mismo dato.

```
const usuarioOriginal = {  
  nombre: "Sara",  
  edad: 30  
};  
  
const usuarioCopia = usuarioOriginal;  
usuarioCopia.edad = 31;  
  
console.log(usuarioOriginal.edad); // 31
```

14.2. Qué ha ocurrido

`usuarioCopia` no es un objeto independiente, sino otra referencia al mismo objeto.

14.3. Copia superficial con `spread`

```
const usuarioOriginal = {  
  nombre: "Sara",  
  edad: 30  
};  
  
const usuarioCopia = { ...usuarioOriginal };  
usuarioCopia.edad = 31;  
  
console.log(usuarioOriginal.edad); // 30  
console.log(usuarioCopia.edad);    // 31
```

14.4. Límite de la copia superficial

Si el objeto contiene datos anidados, la copia con `spread` no aísla completamente las estructuras internas.

```
const usuarioOriginal = {  
  nombre: "Sara",  
  direccion: {  
    ciudad: "Málaga"  
  }  
};
```

```
const usuarioCopia = { ...usuarioOriginal };
usuarioCopia.direccion.ciudad = "Granada";

console.log(usuarioOriginal.direccion.ciudad); // "Granada"
```

14.5. Por qué conviene entenderlo ya

Aunque la inmutabilidad se trabajará más claramente en la sesión siguiente, ya aquí conviene entender que copiar y referenciar no son lo mismo. Esta idea será muy importante después al transformar datos y, más adelante, al trabajar con React.

Referencias específicas del subtema

- MDN Web Docs — Spread syntax
 - MDN Web Docs — Working with objects
-

15. Análisis técnico avanzado

15.1. Arrays y objetos no compiten: se complementan

No debes pensar en array y objeto como opciones rivales. Lo habitual es combinarlos:

- un **objeto** representa una entidad;
- un **array** agrupa varias entidades;
- una propiedad puede contener otro objeto o un array.

15.2. `spread` realiza copia superficial, no profunda

Cuando copias un objeto o un array con `spread`, la copia es superficial.

```
const usuario = {
  nombre: "Sara",
  direccion: {
    ciudad: "Málaga"
  }
};

const copia = { ...usuario };
copia.direccion.ciudad = "Granada";

console.log(usuario.direccion.ciudad); // "Granada"
```

Esto ocurre porque el objeto interno sigue compartiendo referencia.

15.3. optional chaining no debe ocultar mal diseño

`?.` es útil para evitar errores, pero no debe convertirse en una excusa para no entender la estructura real del dato o para aceptar cualquier inconsistencia sin criterio.

15.4. nullish coalescing expresa intención mejor que `||` en muchos casos

Cuando solo quieres cubrir ausencia real de valor, `??` describe mejor tu intención que `||`.

15.5. JSON es central, pero no sustituye al diseño interno

Saber convertir datos a JSON o desde JSON es importante, pero la calidad del programa sigue dependiendo de cómo estructuras esos datos una vez están dentro de tu aplicación.

Referencias específicas del subtema

- MDN Web Docs — Spread syntax
 - MDN Web Docs — Optional chaining (`?.`)
 - MDN Web Docs — Nullish coalescing operator (`??`)
 - MDN Web Docs — JSON
-

16. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```
const peliculas = [
  {
    id: 1,
    titulo: "Arrival",
    anio: 2016,
    disponible: true,
    detalles: {
      duracion: 116,
      director: "Denis Villeneuve"
    }
  },
  {
    id: 2,
    titulo: "Interstellar",
    anio: 2014,
```

```

    disponible: false,
    detalles: {
      duracion: 169,
      director: "Christopher Nolan"
    }
  }
];

const [primeraPelicula] = peliculas;

const {
  titulo,
  disponible,
  detalles: { director, duracion }
} = primeraPelicula;

const estado = disponible ? "Disponible" : "No disponible";
const estudio = primeraPelicula.detalles?.estudio ?? "Estudio no indicado";

console.log(`Título: ${titulo}`);
console.log(`Director: ${director}`);
console.log(`Duración: ${duracion} minutos`);
console.log(`Estado: ${estado}`);
console.log(`Estudio: ${estudio}`);

const copiaPeliculas = [...peliculas];
console.log(copiaPeliculas);

const peliculasJSON = JSON.stringify(peliculas);
console.log(peliculasJSON);

```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
Array de objetos	colección de películas
Datos anidados	objeto <code>detalles</code> dentro de cada película
Desestructuración	extracción de propiedades de la primera película
<code>optional chaining</code>	acceso seguro a una propiedad opcional
<code>nullish coalescing</code>	valor por defecto si falta un dato
<code>spread</code>	copia superficial del array

Elemento	Uso en el ejemplo
JSON	serialización de la colección

17. Errores frecuentes al comenzar

⚠ Errores muy habituales

Estos fallos aparecen con mucha frecuencia al trabajar por primera vez con arrays, objetos y datos anidados.

17.1. Confundir array con objeto

```
const usuario = {
  nombre: "Marta",
  edad: 25
};

// Incorrecto si esperabas un array
console.log(usuario[0]);
```

17.2. Olvidar que los índices empiezan en 0

```
const colores = ["rojo", "verde", "azul"];
console.log(colores[1]); // "verde"
```

Si esperabas "rojo", has olvidado cómo funciona el índice.

17.3. Acceder a propiedades que no existen sin comprobar la ruta

```
const usuario = {
  nombre: "Raquel"
};

// console.log(usuario.direccion.ciudad); // Error
```

17.4. Pensar que `spread` siempre hace una copia completamente independiente

```
const original = {
  nombre: "Iván",
  direccion: {
    ciudad: "Toledo"
  }
};

const copia = { ...original };
copia.direccion.ciudad = "Cuenca";

console.log(original.direccion.ciudad); // "Cuenca"
```

17.5. Usar `||` cuando el valor `0`, `false` o `""` sí son válidos

```
const cantidad = 0;
const mostrada = cantidad || 100;

console.log(mostrada); // 100
```

Si querías conservar `0`, debías considerar `??`.

17.6. Modelar datos con nombres poco claros

```
const producto = {
  a: "Teclado",
  b: 39.99,
  c: true
};
```

Este tipo de estructura dificulta la comprensión y el mantenimiento.

17.7. Creer que dos variables apuntan a datos distintos cuando en realidad comparten referencia

```
const tareasOriginales = ["estudiar", "practicar"];
const tareasCopia = tareasOriginales;

tareasCopia.push("repasar");

console.log(tareasOriginales); // ["estudiar", "practicar", "repasar"]
```

18. Síntesis de la sesión

En esta sesión has trabajado una de las bases estructurales de JavaScript:

- arrays para representar colecciones;
- objetos para representar entidades con propiedades;
- datos anidados como forma habitual de organización real;
- desestructuración para extraer valores con más claridad;
- `spread` y `rest` para copiar, combinar o separar datos;
- `optional chaining` y `nullish coalescing` para acceder con seguridad y establecer valores por defecto;
- JSON como formato fundamental de intercambio de información;
- referencias y copias como idea previa a la transformación inmutable de datos.

También has comenzado a pensar en un aspecto crucial del desarrollo: **cómo modelar bien los datos antes de operar con ellos**.

✓ Al terminar esta sesión deberías ser capaz de...

- distinguir con claridad cuándo usar un array y cuándo usar un objeto;
- acceder y modificar datos simples y anidados;
- recorrer arrays con estructuras básicas;
- utilizar desestructuración en arrays y objetos;
- emplear `spread` y `rest` en escenarios sencillos;
- usar `optional chaining` y `nullish coalescing` con criterio;
- convertir datos entre objeto JavaScript y JSON;
- entender la diferencia básica entre referencia y copia;
- modelar colecciones de información con una estructura razonable.

19. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — Indexed collections
- MDN Web Docs — `Array`
- MDN Web Docs — Working with objects
- MDN Web Docs — Object basics

- MDN Web Docs — Destructuring assignment
- MDN Web Docs — Spread syntax
- MDN Web Docs — Optional chaining (`?.`)
- MDN Web Docs — Nullish coalescing operator (`??`)
- MDN Web Docs — JSON
- MDN Web Docs — `JSON.parse()`
- MDN Web Docs — `JSON.stringify()`

Recomendación de estudio

Al repasar esta sesión, cambia las estructuras de los ejemplos y observa cómo afecta eso al acceso a los datos. Añade propiedades, elimina alguna, introduce objetos dentro de arrays y arrays dentro de objetos, y comprueba qué ocurre. Aprender a modelar datos no consiste solo en leer definiciones, sino en acostumbrarte a ver la estructura de la información con claridad.